

MCP Access Architecture Proposal

Multi-User Scaling & CI/CD Integration

MDC MCP RAG Server — 51 AI-Powered Development Tools
for the NOAA Global Forecast System

Terry McGuinness

Computing & Advanced Technologies (CAT) Unit
Environmental Modeling Center, NCEP/NWS/NOAA
`terry.mcguinness@noaa.gov`

April 30, 2026

1 Executive Summary

The MDC MCP RAG Server provides 51 AI-powered tools for developers working on NOAA's Global Forecast System. These tools query a Neptune graph database (164,916 nodes, 2.9M relationships) and OpenSearch vector store (85,000+ documents) to provide contextual intelligence about the GFS codebase.

This document proposes a **two-phase deployment strategy**:

- **Phase 1:** Internal access for a 10-person cohort via GFE laptops with CAC-enabled VPN, using AgentCore Runtime for per-user session isolation
- **Phase 2:** External access for CI/CD pipelines (GitHub Actions) via FedRAMP-compliant Private API Gateway + Fargate

Both phases maintain **zero internet exposure** for the data plane and comply with NIH/NOAA security requirements.

2 System Overview

MCP Server — 51 Tools

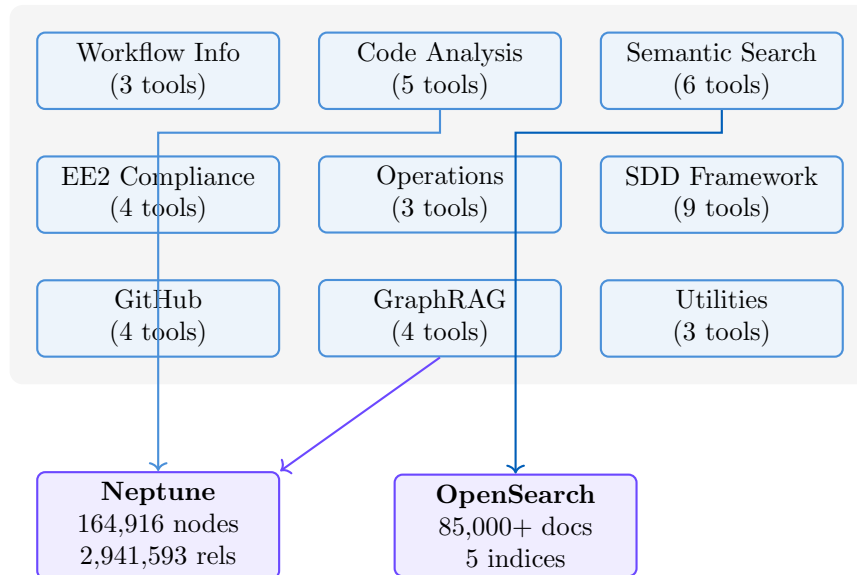


Figure 1: MCP Server tool categories and backend data stores

3 Phase 1: Internal Cohort Access (10 Users)

3.1 Architecture

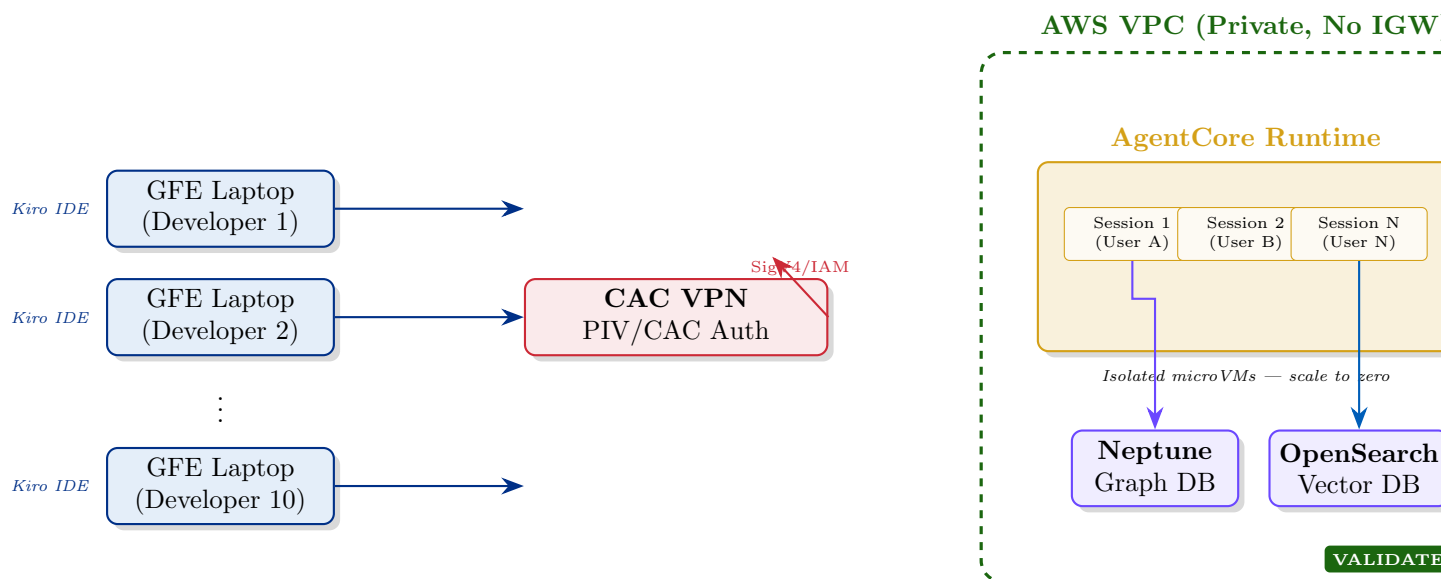


Figure 2: Phase 1 — Internal cohort access via CAC VPN + AgentCore Runtime

3.2 Why AgentCore Runtime

Requirement	AgentCore	Fargate
Per-user session isolation	Dedicated microVM	Shared containers
Scale to zero (no idle cost)	No sessions = \$0	Min 1 task running
No capacity planning	Managed fleet	Must configure scaling
Automatic session cleanup	15 min idle timeout	Must implement
Per-user observability	Session-level X-Ray	Task-level only

Table 1: AgentCore vs. Fargate for internal multi-user access

3.3 Authentication (Validated)

The access pattern is already proven:

1. Developer connects GFE laptop to NOAA VPN (CAC/PIV authentication)
2. Kiro IDE authenticates to AWS via IAM credentials (SigV4)
3. `InvokeAgentRuntime` API call authorized by IAM policy
4. AgentCore creates isolated microVM session for that user
5. Session auto-terminates after 15 minutes idle

Status: First user (developer lead) fully connected and operational as of April 30, 2026.

3.4 AgentCore Runtime (Deployed)

Parameter	Value
Runtime ID	<code>mdc_mcp_rag_server-TMXD11G2Wi</code>
Protocol	MCP (JSON-RPC at <code>:8000/mcp</code>)
Network	VPC (us-east-1a, us-east-1b)
Idle Timeout	900s (15 minutes)
Max Lifetime	28800s (8 hours)
Container	<code>903050880929.dkr.ecr.us-east-1.amazonaws.com/mdc-mcp-rag:agentcore</code>
Status	READY

4 Phase 2: CI/CD Access (GitHub Actions)

4.1 Architecture

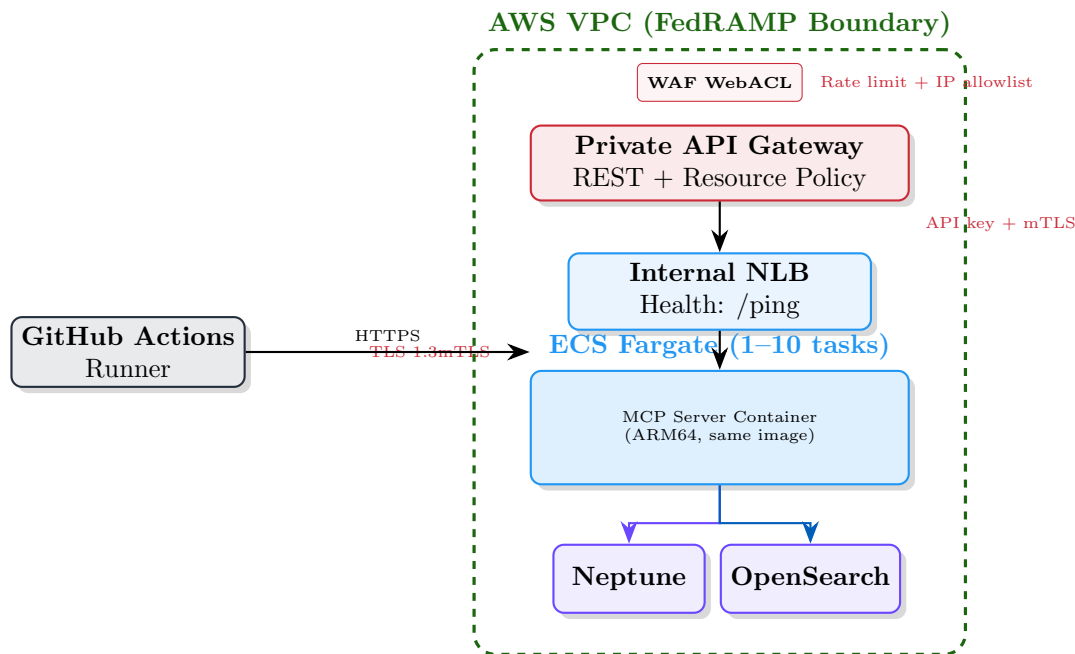


Figure 3: Phase 2 — CI/CD access via Private API Gateway + Fargate (FedRAMP-compliant)

4.2 Why Fargate for Phase 2

Requirement	Fargate	AgentCore
GitHub Actions integration	Standard HTTPS	Requires SDK/proxy
FedRAMP compliance history	Well-established	Newer service
API Gateway + WAF	Native	Not designed for this
Stateless CI requests	Perfect fit	Session overhead
mTLS client certificates	API GW native	Not supported

Table 2: Fargate vs. AgentCore for machine-to-machine CI/CD access

4.3 FedRAMP Controls

Control	Implementation
AC-4 (Information Flow)	Private API GW, VPC endpoint, no IGW
AC-17 (Remote Access)	mTLS + API keys (machines); VPN + CAC (humans)
AU-2 (Audit Events)	CloudWatch Logs for all API GW requests
IA-2 (Identification)	mTLS client certs (machine); Cognito (human)
SC-7 (Boundary Protection)	WAF, resource policy, security groups
SC-8 (Transmission Confidentiality)	TLS 1.3 end-to-end
SC-13 (Cryptographic Protection)	KMS-encrypted at rest (Neptune, OpenSearch)
SI-4 (Monitoring)	X-Ray tracing, CloudWatch alarms, WAF logging

Table 3: FedRAMP security control mapping

5 Phase Comparison

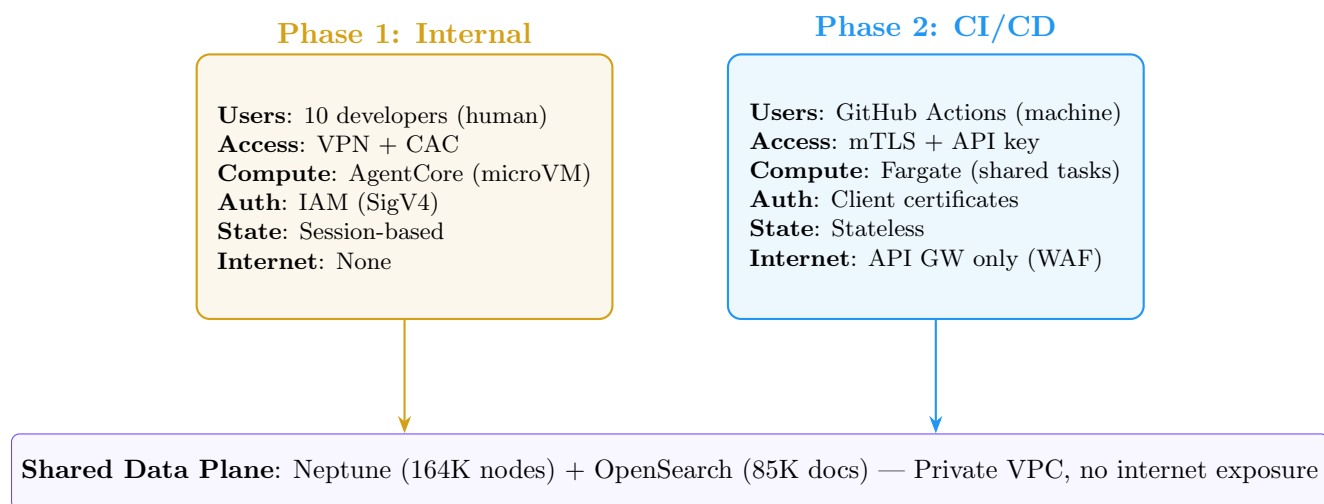


Figure 4: Both phases share the same data plane with zero internet exposure

6 Timeline

Phase	Milestone	Dependencies	Target
1a	AgentCore Runtime deployed	—	Done
1b	First user validated (VPN + IAM + Kiro)	—	Done
1c	9 additional IAM users provisioned	Infra team	May 2026
1d	10-user pilot with Kiro	Onboarding	May 2026
2a	CDK stack deployment (API GW + Fargate)	Infra approval	June 2026
2b	mTLS + WAF configuration	Security team	June 2026
2c	GitHub Actions integration	DevOps	July 2026
2d	FedRAMP documentation update	Compliance	July 2026

Table 4: Deployment timeline

7 Cost Estimates

Component	Usage Model	Est. Monthly
<i>Phase 1 (AgentCore)</i>		
AgentCore compute	10 users × 4 hrs/day × 20 days	\$40
Neptune (baseline)	Already running	\$0 (existing)
OpenSearch (baseline)	Already running	\$0 (existing)
<i>Phase 2 (Fargate + API GW)</i>		
API Gateway	~10K requests/month	\$4
Fargate tasks	1 vCPU, 2GB, auto-scale	\$30
NLB	Hourly + LCU	\$16
WAF	Base + per-request	\$6
Total (both phases)		~\$96/month

Table 5: Monthly cost estimates (excludes existing Neptune/OpenSearch baseline)

8 Existing Assets (Ready to Deploy)

Asset	Location	Status
MCP Server container (ARM64)	ECR: <code>mdc-mcp-rag:agentcore</code>	READY 27 tests
AgentCore Runtime	<code>mdc_mcp_rag_server-TMXD11G2Wi</code>	
CDK Stacks (API GW + Fargate)	<code>infrastructure/cdk/lib/</code>	
Neptune graph database	164,916 nodes, 2.9M rels	
OpenSearch vector store	85,000+ docs, 5 indices	
IAM execution role	<code>mdc-mcp-rag-ecs-task-role</code>	
Service-linked roles	All 4 AgentCore roles	

9 Next Steps

1. **Phase 1c:** Provision 9 additional IAM users with `bedrock-agentcore:InvokeAgentRuntime` permission
2. **Phase 1d:** Onboard developers — each configures Kiro with IAM credentials (same pattern as current setup)
3. **Phase 2a:** Deploy CDK stacks (Private API Gateway + NLB + Fargate) with infra team approval
4. **Phase 2b:** Configure mTLS client certificates and WAF rules for GitHub Actions